Roll No: 2025115140 | HOMEWORK SET 7

C Programming — Data Structures

1. Construct an expression tree for the given infix expression.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

typedef struct Node {
    char data;
    struct Node *left, *right;
} Node;

Node *stack[100];
int top = -1;

void push(Node *x) { stack[++top] = x; }
Node *pop(void) { return stack[top--]; }

Node *newNode(char c) {
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = c;
    p->left = p->right = NULL;
    return p;
}

int prec(char c) {
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

void infixToPostfix(char *in, char *out) {
    char op[100];
    int ot = -1, k = 0;

    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (isalnum((unsigned char)c)) {
            out[k++] = c;
        } else if (c == '(') {
            op[++ot] = c;
        } else if (c == ')') {
            while (ot >= 0 && op[ot] != '(')
                out[k++] = op[ot--];
            if (ot >= 0) ot--;
        } else if (c == '+' || c == '-' || c == '*' || c == '/') {
            while (ot >= 0 && op[ot] != '(' &&
                   prec(op[ot]) >= prec(c))
                out[k++] = op[ot--];
            op[++ot] = c;
        }
    }
    while (ot >= 0) out[k++] = op[ot--];
    out[k] = '\0';
}

Node *buildTree(char *postfix) {
    top = -1;
    for (int i = 0; postfix[i]; i++) {
        char c = postfix[i];
        if (isalnum((unsigned char)c)) {
            push(newNode(c));
        } else {
            Node *r = pop();
            Node *l = pop();
            Node *p = newNode(c);
            p->left = l;
            p->right = r;
            push(p);
        }
    }
    return pop();
}

void preorder(Node *root) {
    if (!root) return;
    printf("%c ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node *root) {
    if (!root) return;
    inorder(root->left);
    printf("%c ", root->data);
    inorder(root->right);
}
int main(void) {
```

```
    char infix[100], postfix[100];
    printf("Enter infix expression: ");
    scanf("%99s", infix);

    infixToPostfix(infix, postfix);
    Node *root = buildTree(postfix);

    printf("Postfix expression: %s\n", postfix);
    printf("Preorder of expression tree: ");
    preorder(root);
    printf("\nInorder of expression tree: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

Sample Input

```
(a-b)*(c+d)
```

Executed Output

```
$ gcc 01_expression_tree.c -o program && ./program
INPUT:
(a-b)*(c+d)

OUTPUT:
Enter infix expression: Postfix expression: ab-cd+*
Preorder of expression tree: * - a b + c d
Inorder of expression tree: a - b * c + d
```

2. Check whether a binary tree is a Binary Search Tree (BST).

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node *newNode(int x) {
    Node *p = (Node *)malloc(sizeof(Node));
    p->data = x;
    p->left = p->right = NULL;
    return p;
}

Node *insert(Node *root, int x) {
    if (!root) return newNode(x);
    if (x < root->data) root->left = insert(root->left, x);
    else root->right = insert(root->right, x);
    return root;
}

int isBST(Node *root, long long low, long long high) {
    if (!root) return 1;
    if (root->data <= low || root->data >= high) return 0;
    return isBST(root->left, low, root->data) &&
           isBST(root->right, root->data, high);
}

int main(void) {
    Node *root = NULL;
    int n, x;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter node values: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }

    if (isBST(root, LLONG_MIN, LLONG_MAX))
        printf("The binary tree is a BST.\n");
    else
        printf("The binary tree is not a BST.\n");

    return 0;
}
```

Sample Input

```
7
50 30 70 20 40 60 80
```

Executed Output

```
$ gcc 02_check_bst.c -o program && ./program
INPUT:
7
50 30 70 20 40 60 80


OUTPUT:
Enter number of nodes: Enter node values: The binary tree is a BST.
```

3. Implement a Max Heap.

C Program

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}

void buildMaxHeap(int a[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
}

int main(void) {
    int a[100], n;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    buildMaxHeap(a, n);

    printf("Max Heap: ");
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    return 0;
}
```

Sample Input

```
7
10 5 20 2 8 15 30
```

Executed Output

```
$ gcc 03_max_heap.c -o program && ./program
INPUT:
7
10 5 20 2 8 15 30


OUTPUT:
Enter number of elements: Enter elements: Max Heap: 30 8 20 2 5 15 10
```

4. Check whether an undirected graph contains a cycle.

C Program

```c
#include <stdio.h>

int graph[50][50], visited[50], n;

int dfs(int u, int parent) {
    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (!graph[u][v]) continue;

        if (!visited[v]) {
            if (dfs(v, u)) return 1;
        } else if (v != parent) {
            return 1;
        }
    }
    return 0;
}

int main(void) {
    int m, u, v;
    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        graph[u][v] = graph[v][u] = 1;
    }

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dfs(i, -1)) {
            printf("Cycle exists in the graph.\n");
            return 0;
        }
    }

    printf("No cycle exists in the graph.\n");
    return 0;
}
```

Sample Input

```
5 5
0 1
1 2
2 3
3 4
4 1
```

Executed Output

```
$ gcc 04_cycle_undirected.c -o program && ./program
INPUT:
5 5
0 1
1 2
2 3
3 4
4 1

OUTPUT:
Enter vertices and edges: Cycle exists in the graph.
```

5. Find the number of islands in a grid.

C Program

```
#include <stdio.h>

char grid[50][50];
int n, m;

void dfs(int r, int c) {
    if (r < 0 || r >= n || c < 0 || c >= m || grid[r][c] != 'L')
        return;

    grid[r][c] = 'W';

    dfs(r + 1, c);
    dfs(r - 1, c);
    dfs(r, c + 1);
    dfs(r, c - 1);
}

int main(void) {
    int islands = 0;

    printf("Enter rows and columns: ");
    scanf("%d%d", &n, &m);

    printf("Enter grid (L = land, W = water):\n");
    for (int i = 0; i < n; i++)
        scanf("%s", grid[i]);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (grid[i][j] == 'L') {
                islands++;
                dfs(i, j);
            }

    printf("Number of islands = %d\n", islands);
    return 0;
}
```

Sample Input

```
5 5
LLWWL
LLWWL
WWLLW
WLLWW
WWWLW
```

Executed Output

```
$ gcc 05_number_of_islands.c -o program && ./program
INPUT:
5 5
LLWWL
LLWWL
WWLLW
WLLWW
WWWLW

OUTPUT:
Enter rows and columns: Enter grid (L = land, W = water):
Number of islands = 4
```

6. Find the Minimum Spanning Tree using Kruskal's algorithm.

C Program

```
#include <stdio.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[100];

int find(int x) {
    if (parent[x] == x) return x;
    return parent[x] = find(parent[x]);
}

void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int main(void) {
    Edge e[200], temp;
    int n, m, cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < m; i++)
        scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

    for (int i = 0; i < m - 1; i++)
        for (int j = i + 1; j < m; j++)
            if (e[i].w > e[j].w) {
                temp = e[i]; e[i] = e[j]; e[j] = temp;
            }

    for (int i = 0; i < n; i++) parent[i] = i;

    printf("Edges in MST:\n");
    for (int i = 0; i < m && count < n - 1; i++) {
        if (find(e[i].u) != find(e[i].v)) {
            unite(e[i].u, e[i].v);
            printf("%d - %d : %d\n", e[i].u, e[i].v, e[i].w);
            cost += e[i].w;
            count++;
        }
    }

    printf("Minimum cost = %d\n", cost);
    return 0;
}
```

Sample Input

```
5 7
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
```

Executed Output

```
$ gcc 06_kruskal_mst.c -o program && ./program
INPUT:
5 7
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9

OUTPUT:
Enter vertices and edges: Enter edges (u v weight):
Edges in MST:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Minimum cost = 16
```

7. Use Dijkstra's algorithm to find shortest paths from a source.

C Program

```c
#include <stdio.h>
#define INF 99999

int main(void) {
    int n, g[50][50], dist[50], used[50] = {0};
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 = no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &g[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &source);

    for (int i = 0; i < n; i++) dist[i] = INF;
    dist[source] = 0;

    for (int k = 0; k < n; k++) {
        int u = -1;

        for (int i = 0; i < n; i++)
            if (!used[i] && (u == -1 || dist[i] < dist[u]))
                u = i;

        if (u == -1 || dist[u] == INF) break;
        used[u] = 1;

        for (int v = 0; v < n; v++)
            if (g[u][v] > 0 && dist[v] > dist[u] + g[u][v])
                dist[v] = dist[u] + g[u][v];
    }

    printf("Shortest distances from %d:\n", source);
    for (int i = 0; i < n; i++)
        printf("Vertex %d = %d\n", i, dist[i]);

    return 0;
}
```

Sample Input

```
5
0 10 3 0 0
0 0 1 2 0
0 4 0 8 2
0 0 0 0 7
0 0 0 9 0
0
```

Executed Output

```
$ gcc 07_dijkstra.c -o program && ./program
INPUT:
5
0 10 3 0 0
0 0 1 2 0
0 4 0 8 2
0 0 0 0 7
0 0 0 9 0
0


OUTPUT:
Enter number of vertices: Enter adjacency matrix (0 = no edge):
Enter source vertex: Shortest distances from 0:
Vertex 0 = 0
Vertex 1 = 7
Vertex 2 = 3
Vertex 3 = 9
Vertex 4 = 5
```

8. Find the mother vertex of a directed graph.

C Program

```
#include <stdio.h>

int g[50][50], visited[50], n;

void dfs(int u) {
    visited[u] = 1;
    for (int v = 0; v < n; v++)
        if (g[u][v] && !visited[v])
            dfs(v);
}

int main(void) {
    int m, u, v, candidate = 0;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        g[u][v] = 1;
    }

    for (int i = 0; i < n; i++)
        if (!visited[i]) {
            dfs(i);
            candidate = i;
        }

    for (int i = 0; i < n; i++) visited[i] = 0;
    dfs(candidate);

    for (int i = 0; i < n; i++)
        if (!visited[i]) {
            printf("No mother vertex exists.\n");
            return 0;
        }

    printf("Mother vertex = %d\n", candidate);
    return 0;
}
```

Sample Input

```
5 6
0 1
0 2
1 3
2 3
3 4
4 0
```

Executed Output

```
$ gcc 08_mother_vertex.c -o program && ./program
INPUT:
5 6
0 1
0 2
1 3
2 3
3 4
4 0

OUTPUT:
Enter vertices and edges: Mother vertex = 0
```

9. Find all Strongly Connected Components using Kosaraju's algorithm.

C Program

```
#include <stdio.h>

int g[50][50], rev[50][50], visited[50], stack[50];
int top = 0, n;

void dfs1(int u) {
    visited[u] = 1;
    for (int v = 0; v < n; v++)
        if (g[u][v] && !visited[v])
            dfs1(v);
    stack[top++] = u;
}

void dfs2(int u) {
    visited[u] = 1;
    printf("%d ", u);
    for (int v = 0; v < n; v++)
        if (rev[u][v] && !visited[v])
            dfs2(v);
}

int main(void) {
    int m, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        g[u][v] = 1;
        rev[v][u] = 1;
    }

    for (int i = 0; i < n; i++)
        if (!visited[i]) dfs1(i);

    for (int i = 0; i < n; i++) visited[i] = 0;

    printf("Strongly Connected Components:\n");
    while (top--) {
        u = stack[top];
        if (!visited[u]) {
            dfs2(u);
            printf("\n");
        }
    }

    return 0;
}
```

Sample Input

```
5 5
0 1
1 2
2 0
1 3
3 4
```

Executed Output

```
$ gcc 09_kosaraju_scc.c -o program && ./program
INPUT:
5 5
0 1
1 2
2 0
1 3
3 4

OUTPUT:
Enter vertices and edges: Strongly Connected Components:
0 2 1
3
4
```

10. Check whether a graph is bipartite.

C Program

```
#include <stdio.h>

int g[50][50], color[50], queue[50], n;

int main(void) {
    int m, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d%d", &n, &m);

    for (int i = 0; i < n; i++) color[i] = -1;

    for (int i = 0; i < m; i++) {
        scanf("%d%d", &u, &v);
        g[u][v] = g[v][u] = 1;
    }

    for (int s = 0; s < n; s++) {
        if (color[s] != -1) continue;

        int front = 0, rear = 0;
        queue[rear++] = s;
        color[s] = 0;

        while (front < rear) {
            u = queue[front++];

            for (v = 0; v < n; v++) {
                if (!g[u][v]) continue;

                if (color[v] == -1) {
                    color[v] = 1 - color[u];
                    queue[rear++] = v;
                } else if (color[v] == color[u]) {
                    printf("Graph is NOT bipartite.\n");
                    return 0;
                }
            }
        }
    }

    printf("Graph is bipartite.\n");
    return 0;
}
```

Sample Input

```
4 4
0 1
0 3
2 1
2 3
```

Executed Output

```
$ gcc 10_bipartite_graph.c -o program && ./program
INPUT:
4 4
0 1
0 3
2 1
2 3


OUTPUT:
Enter vertices and edges: Graph is bipartite.
```